

ECE 436 Project Report II

University of Illinois at Chicago

Date: 4/15/2026

Team 12:

Joshua Garcia, 677345160, jgarc374@uic.edu

Kajus Vysniauskas, 671599156, kvysn2@uic.edu

Muhammed Nedaria, 665610626, mneda2@uic.edu

Adam Krzysztofek, 676346111, akrzy2@uic.edu

OVERVIEW

The experiment examines the real effects that result from performing a SYN flood on a network and the effectiveness of the SYN cookie as an action to defend against the attack. The topology was simplified into a model consisting of a server, client, and host on a Mininet network. With the help of the three phases of execution: baseline, attack, and mitigation, the experiment enabled us to determine the exact way in which malicious packets affect the network's performance and how certain kernel actions can reverse it.

This experiment focuses on the vulnerability of the TCP three-way handshake protocol. Usually, a client will send a SYN packet. Once the SYN packet is received by the server, the server sends a SYN-ACK packet. Once the client receives this SYN-ACK message, the client will respond with an ACK message. A SYN flood attack happens when the server gets overwhelmed with SYN packets. Because of all of these SYN messages, the server never receives the ACK packet from the client causing its memory that it reserved for this to time out. This prevents legitimate users from accessing data from servers.

In order to determine the condition of the network being "healthy", we began by performing twenty connections from H3 to H2 using optimal settings. The capture was performed using Wireshark, and based on these captures, our control metrics were set for throughput, goodput, and latency. During this condition, the network performed optimally, with a 100% success rate and zero delay.

The second phase introduced a high-volume SYN flood initiated by H1. By flooding the server with spoofed SYN packets, each using a randomized source IP, H1 forced the server to exhaust its resources on "half-open" connections that never reached completion. As H3 attempted to navigate this congestion, our data captured a sharp decline in performance. Goodput plummeted as legitimate requests were drowned out by illegitimate traffic, and the latency recorded at H3 spiked, reflecting a server struggling to manage its overwhelmed backlog. The most critical finding here was the significant drop in successful connections to 50%, illustrating the attack's ability to effectively deny service to valid users.

In the final stage, we enabled SYN cookies on the server to test its resilience under continued pressure. This mechanism shifts the server's behavior, allowing it to bypass resource allocation for half-open handshakes until the final ACK is received. Even as H1 maintained its flood, the change in H2's processing logic yielded immediate results. Our packet captures revealed that legitimate packets from H3 were once again handled effectively. While throughput remained high due to the ongoing attack, our goodput stabilized and the connection success rate surged back toward baseline levels of 100%. This shows how significant these mechanisms are in maintaining reliable network services.

As demonstrated in the analysis performed, although a SYN flood attack is quite capable of incapacitating a standard server, SYN cookies form an essential protective measure. From the experiment carried out, it is clear that the attack can effectively disable services due to server backlog exhaustion and an increase in latency to 1096.4 ms from the initial 57.3 ms. Although the use of SYN cookies forms an essential defense mechanism by ensuring 100% connection success, the analysis shows one key trade-off made.

While SYN cookies provide an effective means for mitigating the effects of the attack, the latency associated with the generation and verification of the cookies causes an increase in the delay experienced by legitimate requests by 218.6 ms compared to the baseline scenario. From this analysis, it is evident that SYN cookies can provide effective protection as long as the increased latency can be tolerated by the network.

IMPLEMENTATION

The virtual testing environment used VMware Workstation Pro to create a virtual Linux machine. In this environment, Mininet was used to create a custom network topology that comprised three hosts attached to an Open vSwitch (OVS). Host H1 was the attacking host, H2 was the TCP/HTTP server, while H3 was the genuine client. Each link was assigned a bandwidth of 100 Mbps and a delay of 5 ms.

The small network was set up using a custom python script, topology.py, which created the 3 hosts and the switch and applied critical kernel parameters to H2 at startup. Specifically, H2 was limited to a backlog of 16 to make the server's packet queue fill up quicker during the attack, and SYN cookies were also disabled on start-up for H2. MobaXterm was used alongside the virtual machine to open separate windows for each host making the process

easier to control, and it also allowed for the use of Wireshark within the simulated network to capture all traffic across H2's interface. The experiment carried out 3 core phases. The Baseline phase had H2 run a custom python TCP/HTTP server, server.py, which listened for and accepted connections on port 80. In this phase, H3 ran another custom python script, client_test.py, which makes 20 consecutive HTTP connection requests with H2. Each request carries a 3 second timeout, and the script records the success and latency of each request attempt as well as the average latency (only for successful connections) for the trial and the total successful connections. This phase establishes our normal operating conditions for us to later compare our attack and mitigation phases to.

In the Under Attack phase, H1 launched a SYN flood against H2 using hping3, specifically the command `sudo hping3 -S -flood -rand-source 10.0.0.2 -p 80`. This command sent TCP SYN packets at an estimated rate of 1450 packets per second with randomized spoofed IP addresses effectively flooding H2 and preventing it from completing the 3 way handshake with any flood packet. This causes half-open connections to accumulate and backup the queue. After allowing a couple seconds for the queue to fill, H3 ran the client_test.py script again while the flood continued, measuring the resulting average latency and connection success rate.

In the With Mitigation phase, the flood was kept running and SYN cookies were enabled on H2 by running the command `sysctl -w net.ipv4.tcp_syncookies=1` without shutting down the server. With cookies enabled H3 then ran client_test.py once more to measure the recovery in the latency and connection success rate.

Throughout this process, Wireshark was actively capturing the traffic across H2. TCP flag syn/ack filters and IP address filters were applied to analyze the specific attacking packets coming from H1 as well as the real client packets coming from H3. A thorough analysis was performed at time sections 0-10s, 34-85s, and 101-117s to find the throughput and goodput of the network, as well as informational IO graphs highlighting the effects of the attack.

RESULTS

The experiment was carried out in three phases, Baseline, Under Attack, and With Mitigation, to test the efficiency of SYN cookies as protection from the attack. The measurements were obtained by Wireshark packet capturing and the direct connection monitoring performed on H3's terminal.

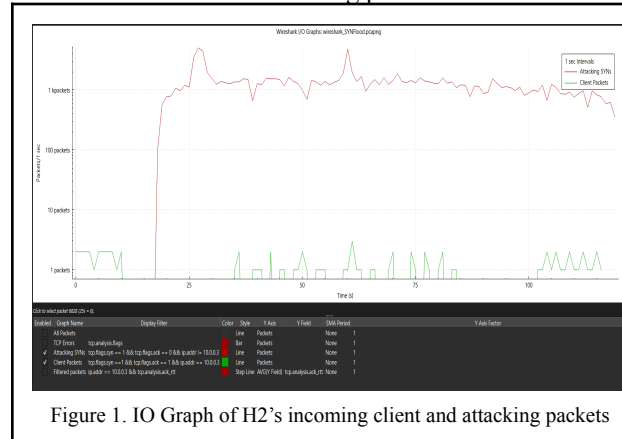


Figure 1. IO Graph of H2's incoming client and attacking packets

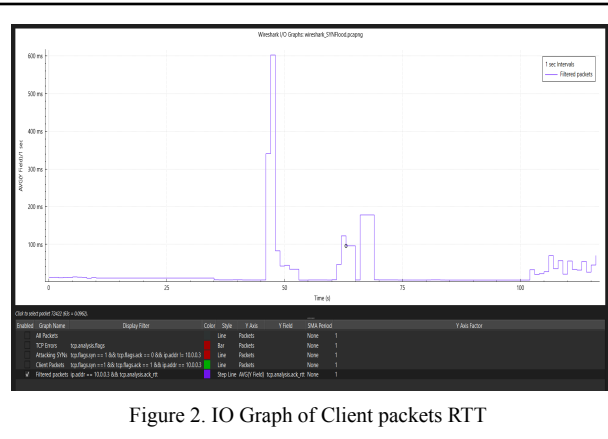


Figure 2. IO Graph of Client packets RTT

During the Baseline phase (0-10s), the network operated normally with no attack traffic present. H3 achieved a 100% connection success rate with an average latency of 57.3ms, a throughput of 11 Kbps, and a goodput of 2.179 Kbps. The IO graph of H2's incoming packets confirms a consistent and steady flow of legitimate client packets during this period, establishing a clean reference point for comparison.

The attack began at approximately 17 seconds, with the Under Attack measurement phase running from 34-85 seconds. During this period, H1 flooded H2 with spoofed SYN packets at a rate of approximately 1,450 SYN/s using hping3. The impact on legitimate traffic was immediate and severe. Throughput exploded to 627 Kbps as the network was saturated with flood traffic, while goodput collapsed to 0.569 Kbps, a 74% reduction in useful data delivery. Connection success rate dropped from 100% to 50%, and average latency increased from 57.3ms to 1,096.4ms, an increase of over 19 times the baseline. The IO graph clearly illustrates this degradation, showing the green client packet flow becoming erratic and spread out as H3 struggled to complete TCP handshakes against a

saturated backlog. The RTT graph shows a sharp upward spike during this phase, confirming the severe impact on response time.

SYN cookies were enabled on H2 at approximately 101 seconds, with the With Mitigation phase measured from 101-117 seconds. Despite the flood remaining active throughout, the results showed a significant recovery. Connection success rate returned fully to 100%, and average latency dropped substantially to 218.6ms. While this is still higher than the 57.3ms baseline, it represents a 80% reduction in latency compared to the attack phase. Goodput recovered to 1.681 Kbps, and the IO graph of incoming packets shows the legitimate client packet flow resuming a consistent pattern resembling the baseline. The RTT graph confirms this recovery with a clear lower average RTT after the 100 second mark.

The residual latency overhead of 161.3ms above baseline during the mitigation phase can be caused by two factors. First, the network was still under active flood during measurement, meaning residual congestion was present. Second, SYN cookie processing introduces a small cryptographic overhead per connection as the server encodes and later verifies the cookie embedded in the TCP sequence number before allocating resources. This tradeoff, accepting a modest latency penalty in exchange for full connection availability under attack, demonstrates that SYN cookies are an effective mitigation strategy. The mechanism successfully defended legitimate traffic without requiring the flood to be stopped, recovering the full connection success rate while keeping latency at a manageable level.

	Throughput (Kbps)	Goodput (Kbps)	Latency (ms)	Connection Success Rate (%)
BASELINE	11	2.179	57.3	100
UNDER_ATTACK	627	0.569	1096.4	50
WITH_MITIGATION	401	1.681	218.6	100

Table 1. Comparison of key statistics between Baseline, Under Attack, and With Mitigation phases

PROTOCOL AND VULNERABILITY ANALYSIS

The core of the described experiment is related to the weakness of the three-way TCP handshake protocol. Normally, the state machine moves from SYN_SENT to SYN_RECV, whereupon the server gets the first SYN message from the client and replies with a SYN-ACK message. In the described “half-open” phase, the server must assign certain memory resources from its SYN backlog to keep track of the connection before receiving the last ACK message from the client. The SYN flood attack makes use of the described vulnerability by flooding the server with a large number of spoofed SYN messages. In the case of the described setup, the SYN backlog limit was 16; hence, all resources were quickly exhausted due to the accumulation of non-established connections.

For the sake of testing the ability of the server to withstand greater pressure, we enabled SYN cookies within the Linux kernel. It makes use of a new approach in which the server is capable of processing half-connections without keeping track of its current status. In other words, instead of allocating a certain amount of memory and storing its connection details, the server uses a special cryptographic hash that contains source IP and port, destination IP and port, as well as a secret timestamp in order to store connection details inside the ISN of the outgoing SYN-ACK segment. The server will allocate physical memory for the connection only when it receives an ACK containing the new cookie.

Even though this type of defense provided 100 percent connectivity success, there was a price paid for such security. As the system needs to calculate a complex algorithm on each arriving SYN packet instead of a simple write into a table of memory, this led to noticeable latency during this process. Thus, the performance degradation caused by the use of "SYN Cookies" is well described by the term used in the paper - the "negative impact". Indeed, the data collected confirmed the effectiveness of the defense mechanism against attacks; however, the latency increased from 57.3 ms to 218.6 ms.

CONCLUSION

In this experiment, it was found that although the SYN flood attack is quite powerful to disable any ordinary server with the help of backlog depletion, SYN cookies play an important role in protecting a network against such attacks. The use of the stateless connection scheme resulted in ensuring 100% success rate of establishing connections despite the attack performed by H1. In other words, clients could easily access the network resources even while they were under DDoS attack. It should be noted, however, that there is a clear trade-off between the network delay and the network availability when dealing with SYN cookies. While being protected from the DDoS attack, the server suffered from increased latency due to the need to calculate and verify each cookie sent. Namely, the average network latency measured 218.6 ms when implementing SYN cookies in comparison with 57.3 ms before using the mitigation technology. It seems to be clear that SYN cookies offer effective protection against the attacks in question only if the network can accept increased latency.

END OF REPORT - Below are extra Documents/Codes/Results:

Used VMware Workstation Pro for linux virtual machine to run mininet. Username/Password: mininet
Used MobaXterm (SSH into it with the IP from the VM) to open up applications like wireshark and the host terminals H1, H2, H3

Simple setup of 3 hosts H1 H2 H3, H1 is the attacker H2 is the TCP server H3 is the client.

ATTACK SCENARIO - SYN Flood Attack on a TCP Server

MITIGATION TECHNIQUE - SYN Cookies

METRICS - Throughput/Goodput, Connection Success Rate, Latency

EXTENSION - Impact of security mechanisms on legitimate traffic

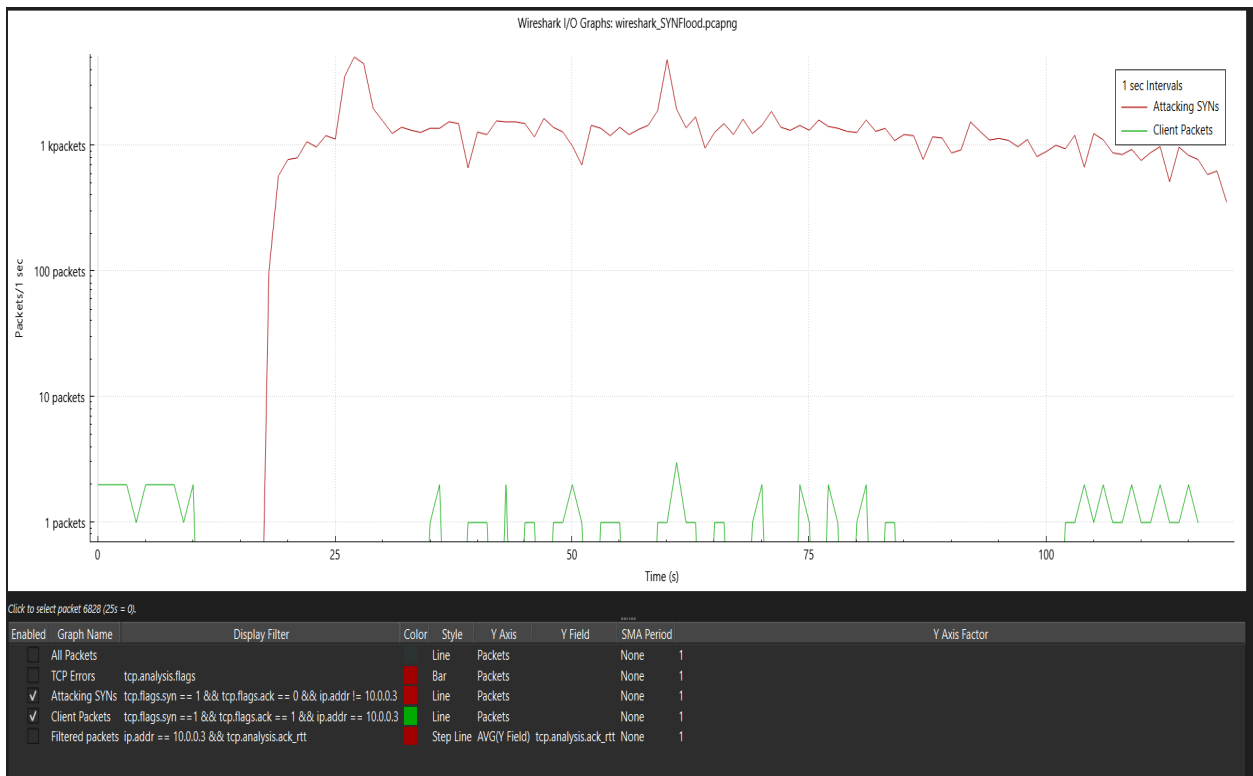
FINDINGS/STATISTICS - Below are the key findings from the simulation. Two graphs depicting the effect SYN cookies had on mitigating the attack. A table comparing key metrics for each scenario, BASELINE, UNDER_ATTACK, and WITH_MITIGATION. Statistics were gathered through Wireshark as well as tracking the latency and connection success rate through the clients xterm terminal H3.

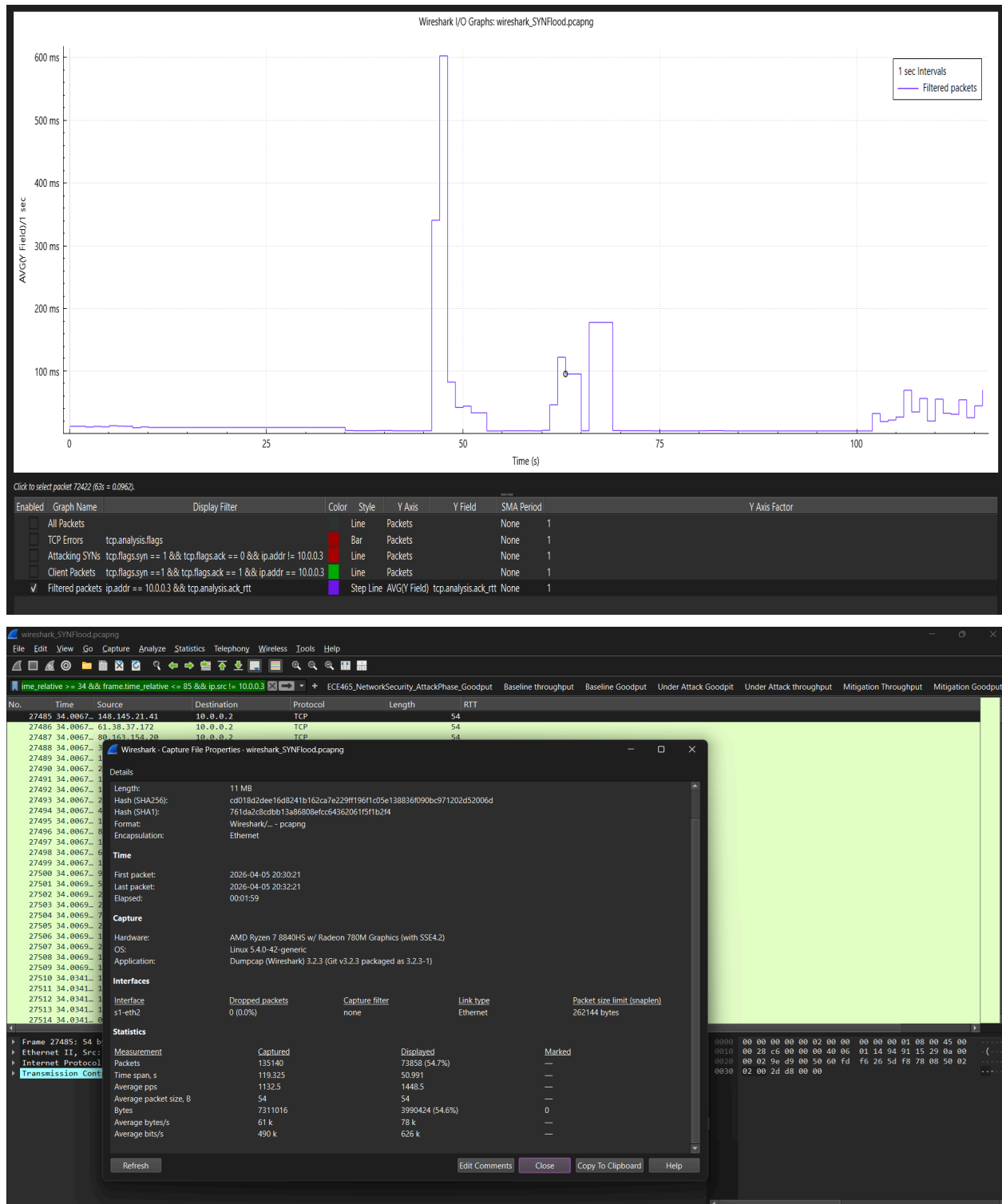
Baseline: 0-10s

Under_attack: 34-85s

Mitigation:101-117s

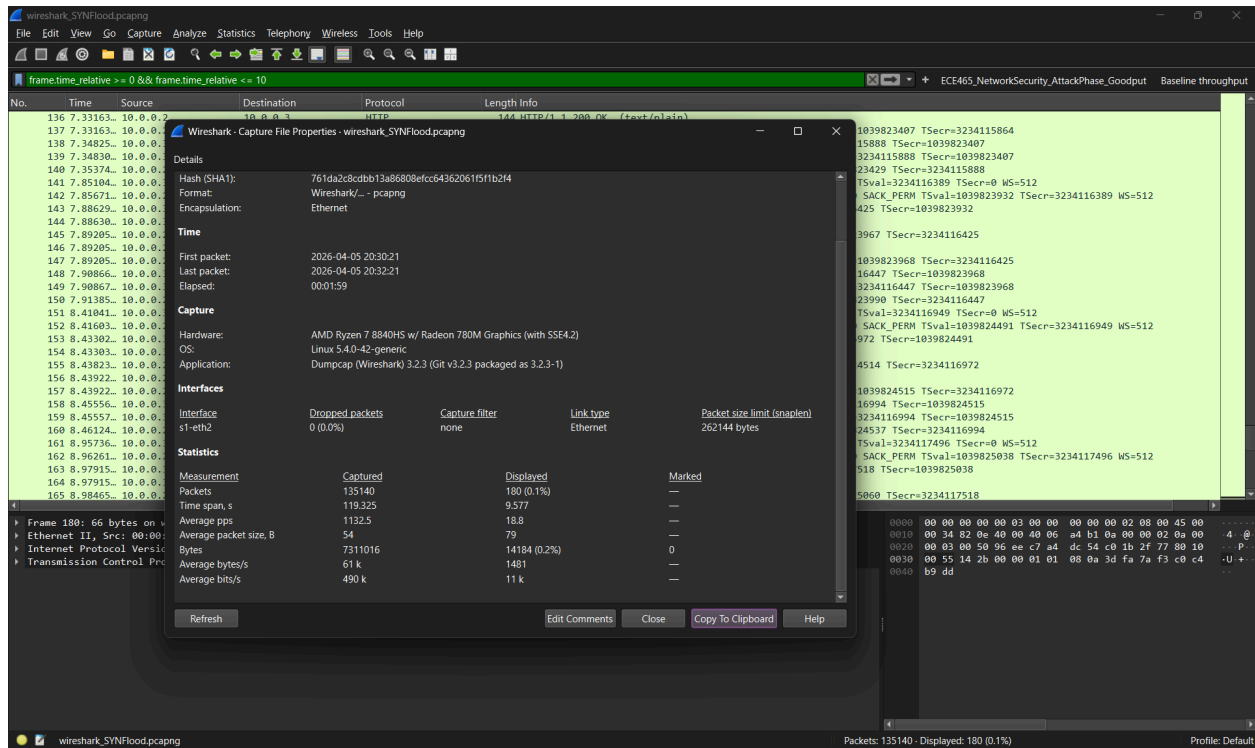
Graph of attacking and client packets from H2’s perspective



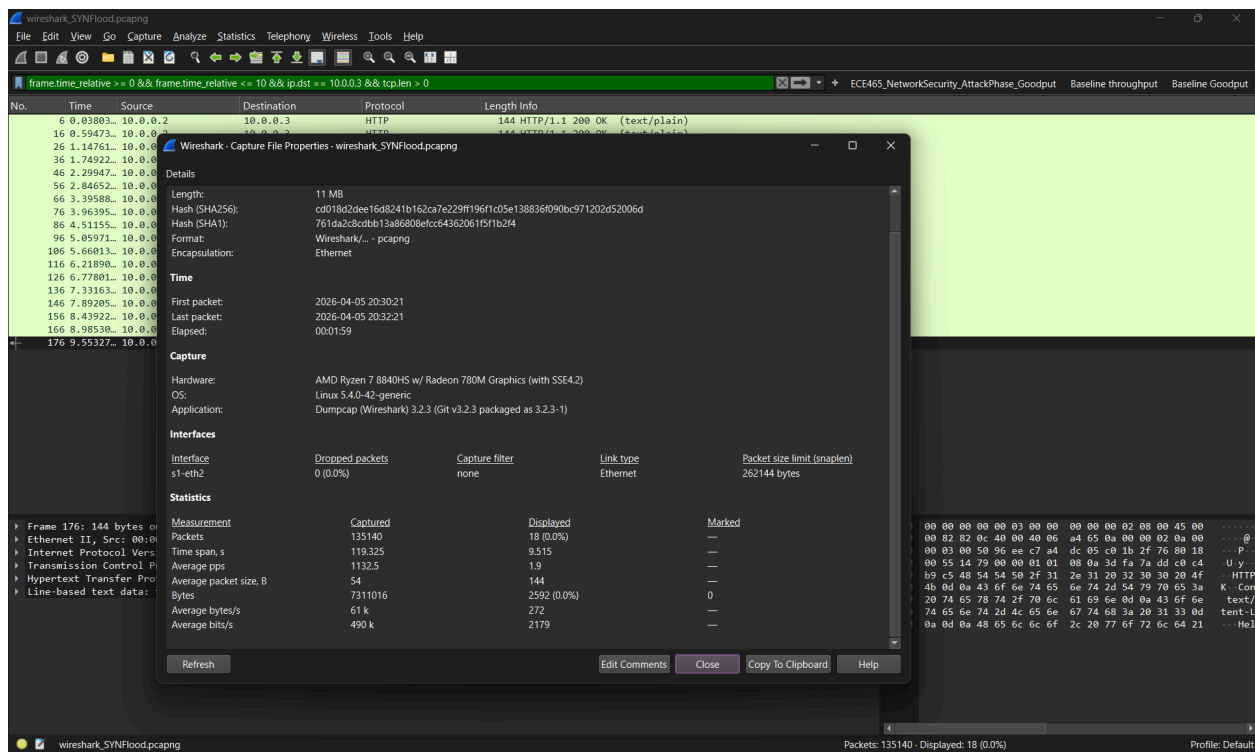


The picture above shows statistics of the packets sent to H2 during the time period 34-85 seconds that did NOT come from H3. We can see the average is 1448.5 packets/second. This is the rate of the spoofed SYN packets flooding H2.

BASELINE 0-10 seconds

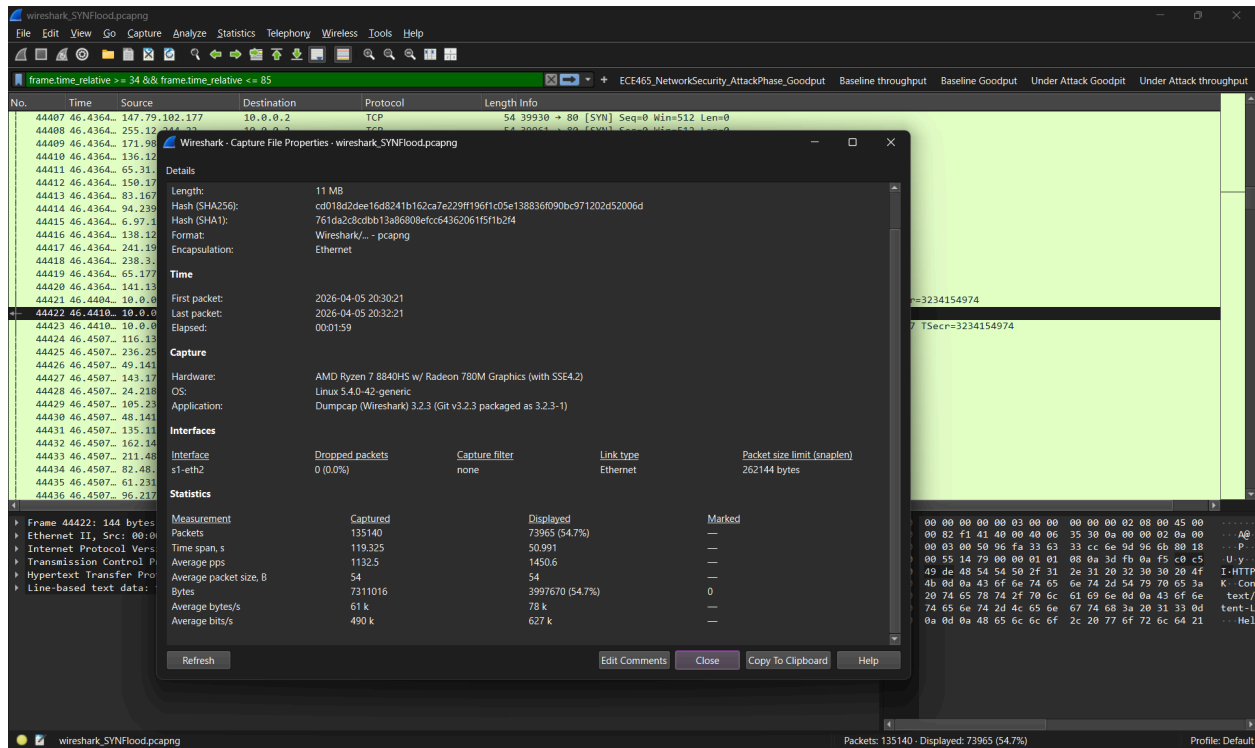


Throughput: 11Kbps

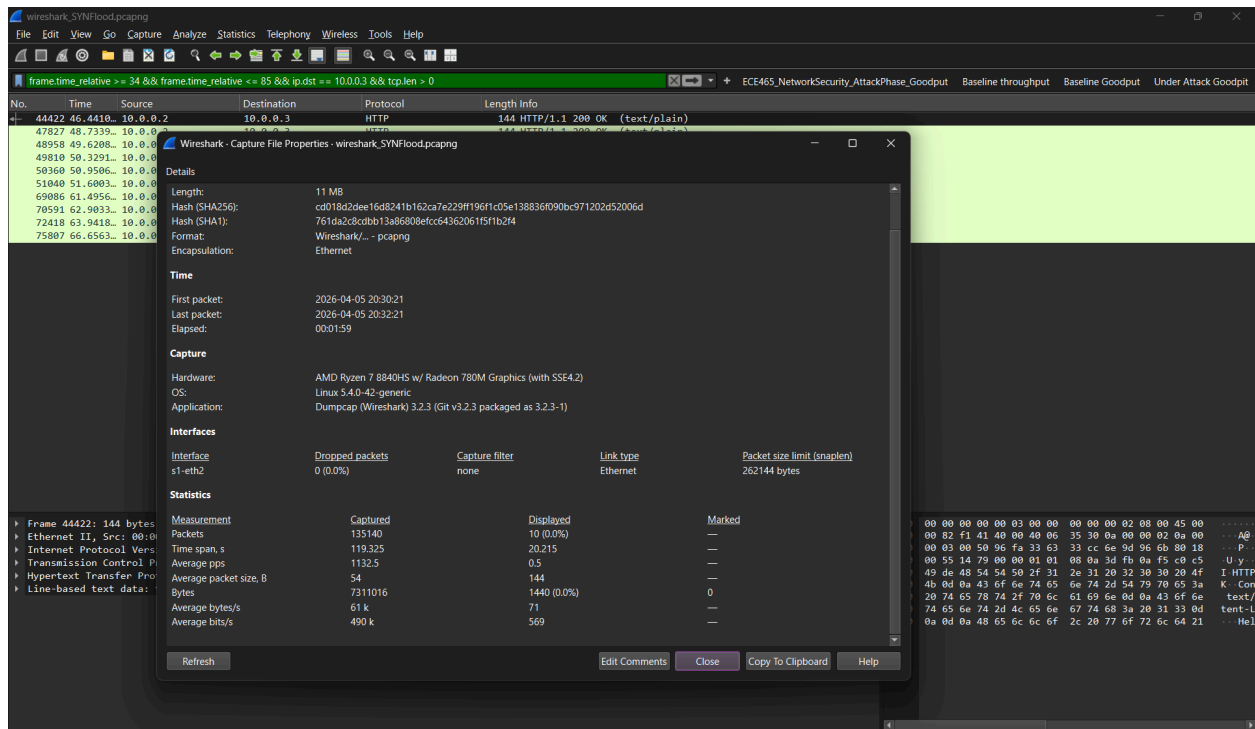


Goodput: 2.179Kbps

UNDER ATTACK 34-85 seconds

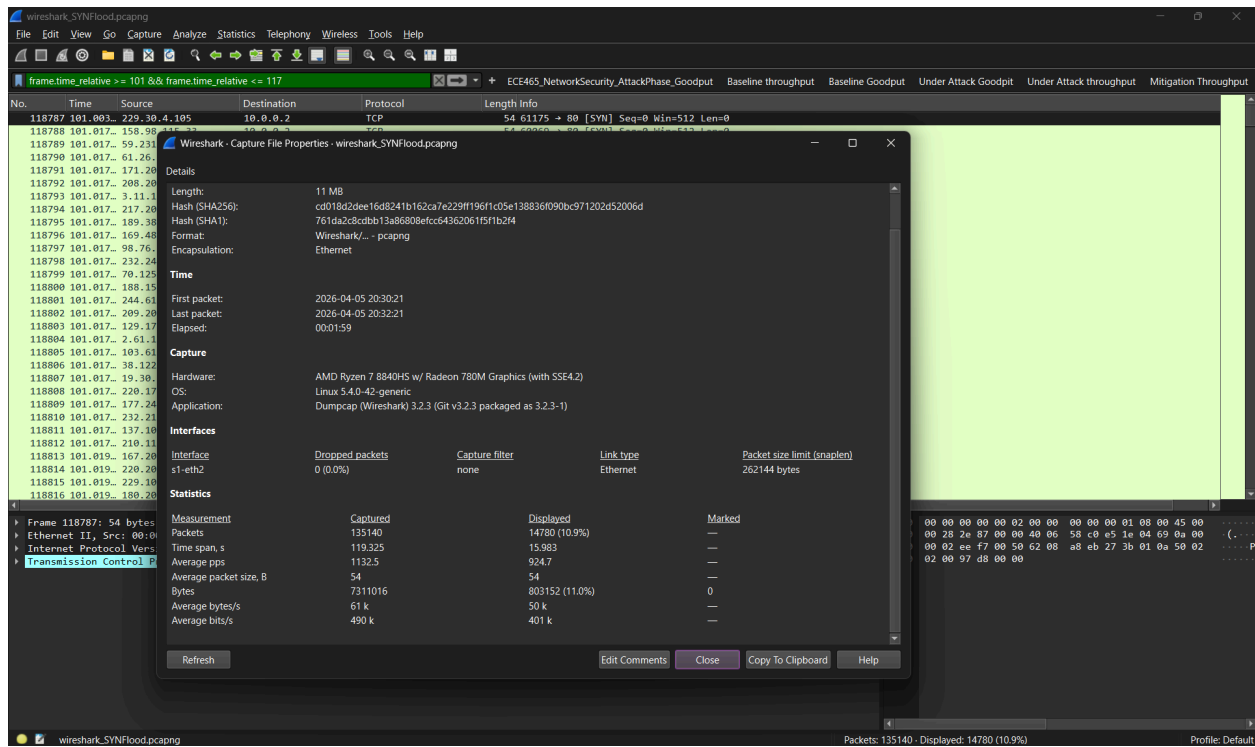


Throughput: 627Kbps

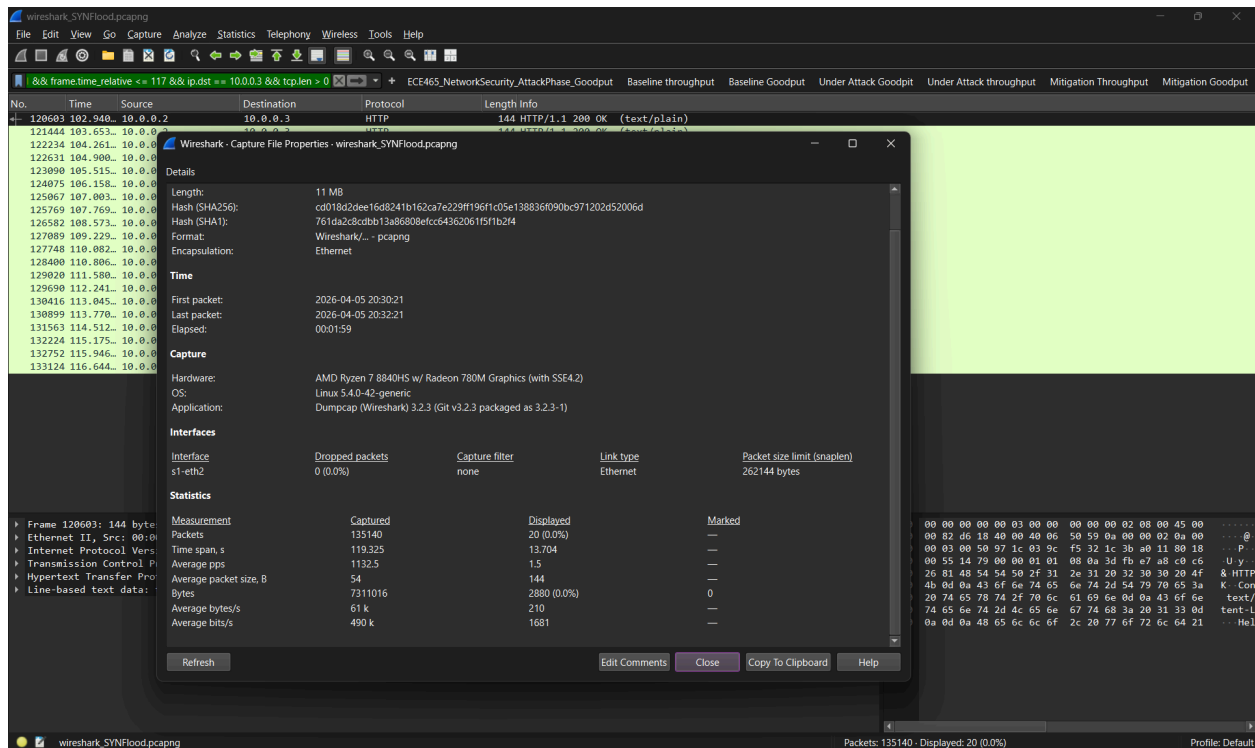


Goodput: 0.569Kbps

MITIGATION 101-117 seconds



Throughput: 401Kbps



Goodput: 1.681Kbps

LATENCY (RTT) OVER TIME

Executables/Code:

Below are our executables/codes used in the project. There is also a simple guide provided for a brief walkthrough and how to perform the experiment using mininet.

GUIDE

```
"""
Automates the entire experiment from one Mininet CLI session.
Paste each xterm command block into the right terminal.
"""

GUIDE = """
=====

    SYN FLOOD + SYN COOKIE MITIGATION - FULL GUIDE
=====

SETUP (run once in Mininet CLI or from your host VM):
-----

pip3 install scapy      # on h1 / h2 / h3 as needed

STEP 0a - Start Mininet
sudo xauth add $(xauth list $DISPLAY)
- needed for some reason to display xterm terminals h1 h2 h3, probably due to permissions
sudo python3 topology.py

STEP 0b - Open terminal windows for each host in Mininet CLI:
xterm h1 h2 h3
(MobaXterm: each will open as a separate tab)
Additionally you can open multiple host windows, which will be helpful later for enabling cookies for H2

=====

PHASE 1: BASELINE (no attack) cookies OFF
=====

-- H2 TERMINAL
python3 server.py
-Watch for "Listening on 0.0.0.0:80"

-- H3 TERMINAL
python3 client_test.py --label BASELINE --tries 20
-Baseline: estimated 100% success rate

```

PHASE 2: UNDER ATTACK (SYN cookies OFF)

-- H2 TERMINAL

sysctl net.ipv4.tcp_syncookies

-Confirm: tcp_syncookies: 0 (Cookies DISABLED)

-- H1 TERMINAL

sudo hping3 -S --flood --rand-source 10.0.02 -p 80

-Wait a few seconds for backlog to fill

-- H3 TERMINAL (while attack running)

python3 client_test.py --label UNDER_ATTACK --tries 20

-Should see TIMEOUT errors

-Success rate will drop significantly

PHASE 3: WITH MITIGATION (SYN cookies ON)

-- H1 TERMINAL

Keep flood running

-- H2 TERMINAL

sysctl -w net.ipv4.tcp_syncookies=1

-This enables SYN cookies

-- H3 TERMINAL (while attack still running)

python3 client_test.py --label WITH_MITIGATION --tries 20

-Should expect to see more OK connections

-Latency could be higher than Baseline

if __name__ == '__main__':
 print(GUIDE)

topology.py - run to start up mininet and setup the small 3 hosts network

```
from mininet.net import Mininet
from mininet.node import OVSSwitch, Controller
from mininet.link import TCLink
from mininet.log import setLogLevel
from mininet.cli import CLI

def build_topology():
    setLogLevel('info')

    # create the network
    net = Mininet(switch=OVSSwitch, controller=Controller, link=TCLink)

    net.addController('c0')
    s1 = net.addSwitch('s1')

    # h1 = attacker, h2 = server, h3 = legitimate client
    h1 = net.addHost('h1', ip='10.0.0.1/24', mac='00:00:00:00:00:01')
    h2 = net.addHost('h2', ip='10.0.0.2/24', mac='00:00:00:00:00:02')
    h3 = net.addHost('h3', ip='10.0.0.3/24', mac='00:00:00:00:00:03')

    # connect each host to the switch with 100Mbps links and 5ms delay
    net.addLink(h1, s1, bw=100, delay='5ms')
    net.addLink(h2, s1, bw=100, delay='5ms')
    net.addLink(h3, s1, bw=100, delay='5ms')

    net.start()

    # lower the SYN backlog on h2 so the attack fills it faster
    # also disable SYN cookies at start so we can show the attack working first
    h2.cmd('sysctl -w net.ipv4.tcp_max_syn_backlog=16')
    h2.cmd('sysctl -w net.core.somaxconn=16')
    h2.cmd('sysctl -w net.ipv4.tcp_syncookies=0')

    print("Topology ready. Hosts: h1 (attacker), h2 (server), h3 (client)")
    print("")
    print('Steps:')
    print(' 1. h2: python3 server.py')
    print(' 2. h3: python3 client_test.py --label BASELINE --tries 20')
```

```

print(' 3. h1: hping3 -S --flood --rand-source 10.0.0.2 -p 80')
print(' 4. h3: python3 client_test.py --label UNDER_ATTACK --tries 20')
print(' 5. h2: sysctl -w net.ipv4.tcp_syncookies=1')
print(' 6. h3: python3 client_test.py --label WITH_MITIGATION --tries 20')
print("")

CLI(net)
net.stop()

if __name__ == '__main__':
    build_topology()

```

[server.py](#) - run in H2 xterm terminal. This creates a simple TCP/HTTP server that accepts connections and returns back a simple “Hello World!” We also print the connection attempt success on H2’s terminal as well as periodically print the SYN cookies status to display whether cookies have been enabled or not. To enable mitigation, aka enable cookies, we run the command `sudo sysctl -w net.ipv4.tcp_syncookies=1` in a separate H2 xterm terminal.

```

"""
Simple TCP/HTTP server on H2 (10.0.0.2)
"""
import socket
import threading
import datetime
import time

HOST = '0.0.0.0'
PORT = 80
BACKLOG = 16
connection_count = 0
lock = threading.Lock()

def log(msg):
    ts = datetime.datetime.now().strftime('%H:%M:%S.%f')[:-3]
    print(f"[{ts}] {msg}", flush=True)

def syn_cookie_status():
    with open('/proc/sys/net/ipv4/tcp_syncookies') as f:
        val = f.read().strip()
        return 'ON' if val == '1' else 'OFF'

```

```

def status_printer():
    while True:
        time.sleep(2)
        print(f'SYN-cookies: {syn_cookie_status()}\n', flush=True)

def handle_client(conn, addr):
    global connection_count
    with lock:
        connection_count += 1
        count = connection_count
    log(f'[OK] CONNECTION #{count} from {addr[0]}:{addr[1]}')
    try:
        data = conn.recv(1024)
        response = (
            b"HTTP/1.1 200 OK\r\n"
            b"Content-Type: text/plain\r\n"
            b"Content-Length: 13\r\n"
            b"\r\n"
            b"Hello, world!"
        )
        conn.sendall(response)
    except:
        pass
    finally:
        conn.close()

def main():
    with open('/proc/sys/net/ipv4/tcp_max_syn_backlog', 'r') as f:
        backlog_limit = f.read().strip()
    log(f'Server starting on port {PORT}')
    cookie_text = 'DISABLED'
    if syn_cookie_status().startswith('ON'):
        cookie_text = 'ENABLED'
    log(f'SYN cookies: {cookie_text}')
    log(f'SYN backlog limit: {backlog_limit}')
    log(f'Listening on {HOST}:{PORT} (backlog={BACKLOG})')
    status_t = threading.Thread(target=status_printer, daemon=True)
    status_t.start()

```

```

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((HOST, PORT))
    s.listen(BACKLOG)
    while True:
        try:
            conn, addr = s.accept()
            client_t = threading.Thread(target=handle_client, args=(conn, addr))
            client_t.daemon = True
            client_t.start()
        except KeyboardInterrupt:
            log("Server stopped.")
            break
if __name__ == '__main__':
    main()

```

Client_test.py - run in H3 xterm terminal. This is a simple code that has H3 try and connect to H2 20 consecutive times. H3 tracks its latency for each connection request, whether that request succeeded or failed and potentially why (timeout, refusal, etc). H3 prints this information for each request to the terminal for user experience, and at the end calculates the average latency and connection success rate for the trial. This script also allows users to pass in arguments “label”, “tries”, and “timeout”. For example, the user can run `python3 client_test.py -label TESTING -tries 100 -timeout 10`

which will then have H3 try to connect to H2 100 times with a timeout period of 10 seconds and “label” the trial “TESTING”. This label is only used for printing to the terminal to differentiate trails from each other but can easily be used to store the results in different files and differentiate them by the label being passed. The default timeout is 3 seconds, the default tries is 20, and the default label is TEST.

```

"""
Real Client on H3 (10.0.0.3)
"""
import socket
import time
import datetime
import sys
import argparse
SERVER_IP = '10.0.0.2'

```

```

SERVER_PORT = 80
TIMEOUT     = 3.0
NUM_TRIES   = 20
DELAY       = 0.5

def try_connect():
    start = time.time()
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(TIMEOUT)
    try:
        s.connect((SERVER_IP, SERVER_PORT))
        s.sendall(b"GET / HTTP/1.0\r\nHost: 10.0.0.2\r\n\r\n")
        data = s.recv(256)
        latency = (time.time() - start) * 1000
        success = len(data) > 0
        return success, latency, None
    except socket.timeout:
        latency = (time.time() - start) * 1000
        return False, latency, "TIMEOUT"
    except ConnectionRefusedError:
        latency = (time.time() - start) * 1000
        return False, latency, "REFUSED"
    except Exception as e:
        latency = (time.time() - start) * 1000
        return False, latency, str(e)
    finally:
        s.close()

def run_test(label):
    print("\n" + "=" * 55)
    print(f" CLIENT TEST - {label}")
    print(f" Target : {SERVER_IP}:{SERVER_PORT}")
    print(f" Tries: {NUM_TRIES} | Timeout: {TIMEOUT}s")
    print("=" * 55)
    successes = 0
    timeouts = 0
    refusals = 0
    latencies = []

```

```

for i in range(1, NUM_TRIES + 1):
    ts = datetime.datetime.now().strftime('%H:%M:%S.%f')[:-3]
    success, latency, error = try_connect()
    if success:
        successes += 1
        latencies.append(latency)
        status = f"[OK] SUCCESS    {latency:7.1f} ms"
    else:
        if error == "TIMEOUT":
            timeouts += 1
        elif error == "REFUSED":
            refusals += 1
        status = f"[X] {error:<11} {latency:7.1f} ms"

    print(f" [{ts}] #{i:02d} {status}", flush=True)
    time.sleep(DELAY)

success_rate = successes / NUM_TRIES * 100
avg_lat = 0
min_lat = 0
max_lat = 0
if latencies:
    avg_lat = sum(latencies) / len(latencies)
    min_lat = min(latencies)
    max_lat = max(latencies)

print("\n" + "-" * 55)
print(f" RESULTS for: {label}")
print("-" * 55)
print(f" Total attempts    : {NUM_TRIES}")
print(f" Successes         : {successes} ( {success_rate:.1f}%)")
print(f" Timeouts          : {timeouts}")
print(f" Connection refused : {refusals}")
print(f" Avg latency       : {avg_lat:.1f} ms")
print(f" Min / Max latency  : {min_lat:.1f} / {max_lat:.1f} ms")
print("=" * 55 + "\n")

if __name__ == '__main__':
    parser = argparse.ArgumentParser()

```

```

parser.add_argument('--label', default='TEST')
parser.add_argument('--tries', type=int, default=NUM_TRIES)
parser.add_argument('--timeout', type=float, default=TIMEOUT)
args = parser.parse_args()
NUM_TRIES = args.tries
TIMEOUT = args.timeout
run_test(args.label)

```

There is no custom python script for H1, instead we run the command:

sudo hping3 -S --flood --rand-source 10.0.0.2 -p 80

Which starts sending SYN packets with randomized spoofed IP's to 10.0.0.2 on port 80 corresponding to the server running on H2

Terminal Outputs:

Below are the terminal outputs, starting with the main VM terminal in MobaXterm and then the 3 hosts terminals. In bold is the user input line with the result copied from the terminal below.

VM Terminal in MobaXterm

```

mininet@mininet-vm:~/ECE436_networksecurity_project$ sudo python3 topology.py
(100.00Mbit 5ms delay) (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) *** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) ... (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) (100.00Mbit 5ms delay)
Topology ready. Hosts: h1 (attacker), h2 (server), h3 (client)

Steps:
 1. h2: python3 server.py
 2. h3: python3 client_test.py --label BASELINE --tries 20
 3. h1: hping3 -S --flood --rand-source 10.0.0.2 -p 80
 4. h3: python3 client_test.py --label UNDER_ATTACK --tries 20
 5. h2: sysctl -w net.ipv4.tcp_syncookies=1
 6. h3: python3 client_test.py --label WITH_MITIGATION --tries 20

*** Starting CLI:
mininet>

```

mininet@mininet-vm:~/ECE436_networksecurity_project\$ sudo python3 topology.py

(100.00Mbit 5ms delay) (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) *** Configuring hosts

h1 h2 h3

*** Starting controller

c0

*** Starting 1 switches

s1 (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) ... (100.00Mbit 5ms delay) (100.00Mbit 5ms delay) (100.00Mbit 5ms delay)

Topology ready. Hosts: h1 (attacker), h2 (server), h3 (client)

Steps:

1. h2: python3 server.py
2. h3: python3 client_test.py --label BASELINE --tries 20
3. h1: hping3 -S --flood --rand-source 10.0.0.2 -p 80

4. h3: python3 client_test.py --label UNDER_ATTACK --tries 20
5. h2: sysctl -w net.ipv4.tcp_syncookies=1
6. h3: python3 client_test.py --label WITH_MITIGATION --tries 20

*** Starting CLI:

TERMINAL OUTPUT for the SERVER (H2), CLIENT (H3), and ATTACKER (H1) These are NOT in order of execution:

H1 - ATTACKER

```
root@mininet-vm:/home/mininet/ECE436_networksecurity_project# sudo hping3 -S --flood --rand-source 10.0.0.2 -p 80
```

```
HPING 10.0.0.2 (h1-eth0 10.0.0.2): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

```
--- 10.0.0.2 hping statistic ---
686245 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

```
root@mininet-vm:/home/mininet/ECE436_networksecurity_project# sudo hping3 -S --flood --rand-source 10.0.0.2 -p 80
```

```
HPING 10.0.0.2 (h1-eth0 10.0.0.2): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

```
--- 10.0.0.2 hping statistic ---
258849 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

```
root@mininet-vm:/home/mininet/ECE436_networksecurity_project# sudo hping3 -S --flood --rand-source 10.0.0.2 -p 80
```

```
HPING 10.0.0.2 (h1-eth0 10.0.0.2): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

```
--- 10.0.0.2 hping statistic ---
1137298 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

H3 - CLIENT

```
root@mininet-vm:/home/mininet/ECE436_networksecurity_project# python3 client_test.py --label BASELINE --tries 20
```

CLIENT TEST - BASELINE

Target : 10.0.0.2:80

Tries: 20 | Timeout: 3.0s

[18:30:21.671] #01	[OK]	SUCCESS	54.6 ms
[18:30:22.226] #02	[OK]	SUCCESS	56.7 ms
[18:30:22.785] #03	[OK]	SUCCESS	50.8 ms

```
[18:30:23.336] #04 [OK] SUCCESS 104.0 ms
[18:30:23.942] #05 [OK] SUCCESS 45.4 ms
[18:30:24.488] #06 [OK] SUCCESS 47.5 ms
[18:30:25.037] #07 [OK] SUCCESS 46.6 ms
[18:30:25.585] #08 [OK] SUCCESS 66.6 ms
[18:30:26.153] #09 [OK] SUCCESS 47.9 ms
[18:30:26.702] #10 [OK] SUCCESS 44.8 ms
[18:30:27.249] #11 [OK] SUCCESS 100.0 ms
[18:30:27.851] #12 [OK] SUCCESS 56.4 ms
[18:30:28.408] #13 [OK] SUCCESS 57.6 ms
[18:30:28.967] #14 [OK] SUCCESS 53.3 ms
[18:30:29.521] #15 [OK] SUCCESS 58.6 ms
[18:30:30.081] #16 [OK] SUCCESS 45.8 ms
[18:30:30.628] #17 [OK] SUCCESS 44.6 ms
[18:30:31.174] #18 [OK] SUCCESS 67.0 ms
[18:30:31.743] #19 [OK] SUCCESS 52.5 ms
[18:30:32.296] #20 [OK] SUCCESS 46.0 ms
```

RESULTS for: BASELINE

Total attempts : 20
Successes : 20 (100.0%)
Timeouts : 0
Connection refused : 0
Avg latency : 57.3 ms
Min / Max latency : 44.6 / 104.0 ms

root@mininet-vm:/home/mininet/ECE436_networksecurity_project# python3 client_test.py --label UNDER_ATTACK --tries 20

CLIENT TEST - UNDER_ATTACK

Target : 10.0.0.2:80
Tries: 20 | Timeout: 3.0s

```
[18:30:55.349] #01 [X] TIMEOUT 3001.9 ms
[18:30:58.852] #02 [X] TIMEOUT 3002.8 ms
[18:31:02.357] #03 [X] TIMEOUT 3003.5 ms
[18:31:05.862] #04 [OK] SUCCESS 3150.8 ms
[18:31:09.514] #05 [OK] SUCCESS 911.6 ms
[18:31:10.927] #06 [OK] SUCCESS 400.3 ms
[18:31:11.828] #07 [OK] SUCCESS 212.2 ms
[18:31:12.542] #08 [OK] SUCCESS 195.2 ms
[18:31:13.238] #09 [OK] SUCCESS 51.1 ms
[18:31:13.790] #10 [X] TIMEOUT 3001.4 ms
[18:31:17.293] #11 [X] TIMEOUT 3001.8 ms
[18:31:20.796] #12 [OK] SUCCESS 2520.1 ms
```

```
[18:31:23.818] #13 [OK] SUCCESS    1146.7 ms
[18:31:25.466] #14 [OK] SUCCESS    167.1 ms
[18:31:26.135] #15 [OK] SUCCESS   2209.2 ms
[18:31:28.845] #16 [X] TIMEOUT   3001.7 ms
[18:31:32.348] #17 [X] TIMEOUT   3002.1 ms
[18:31:35.852] #18 [X] TIMEOUT   3003.4 ms
[18:31:39.356] #19 [X] TIMEOUT   3002.4 ms
[18:31:42.860] #20 [X] TIMEOUT   3002.2 ms
```

RESULTS for: UNDER_ATTACK

Total attempts : 20
Successes : 10 (50.0%)
Timeouts : 10
Connection refused : 0
Avg latency : 1096.4 ms
Min / Max latency : 51.1 / 3150.8 ms

**root@mininet-vm:/home/mininet/ECE436_networksecurity_project# python3 client_test.py --label
WITH_MITIGATION --tries 20**

CLIENT TEST - WITH_MITIGATION

Target : 10.0.0.2:80
Tries: 20 | Timeout: 3.0s

```
[18:32:04.495] #01 [OK] SUCCESS    158.3 ms
[18:32:05.154] #02 [OK] SUCCESS    192.2 ms
[18:32:05.848] #03 [OK] SUCCESS    129.4 ms
[18:32:06.478] #04 [OK] SUCCESS    118.6 ms
[18:32:07.098] #05 [OK] SUCCESS    116.5 ms
[18:32:07.715] #06 [OK] SUCCESS    299.0 ms
[18:32:08.515] #07 [OK] SUCCESS    199.6 ms
[18:32:09.215] #08 [OK] SUCCESS    246.1 ms
[18:32:09.962] #09 [OK] SUCCESS    323.5 ms
[18:32:10.786] #10 [OK] SUCCESS    145.7 ms
[18:32:11.433] #11 [OK] SUCCESS    348.2 ms
[18:32:12.282] #12 [OK] SUCCESS    265.0 ms
[18:32:13.048] #13 [OK] SUCCESS    235.0 ms
[18:32:13.785] #14 [OK] SUCCESS    176.3 ms
[18:32:14.462] #15 [OK] SUCCESS    289.0 ms
[18:32:15.252] #16 [OK] SUCCESS    262.3 ms
[18:32:16.015] #17 [OK] SUCCESS    201.0 ms
[18:32:16.717] #18 [OK] SUCCESS    248.7 ms
[18:32:17.467] #19 [OK] SUCCESS    176.6 ms
[18:32:18.144] #20 [OK] SUCCESS    240.9 ms
```

RESULTS for: WITH_MITIGATION

Total attempts : 20
Successes : 20 (100.0%)
Timeouts : 0
Connection refused : 0
Avg latency : 218.6 ms
Min / Max latency : 116.5 / 348.2 ms

H2 - SERVER

Starting the server

root@mininet-vm:/home/mininet/ECE436_networksecurity_project# python3 server.py

[18:04:48.738] Server starting on port 80

[18:04:48.753] SYN cookies: DISABLED

[18:04:48.753] SYN backlog limit: 16

[18:04:48.755] Listening on 0.0.0.0:80 (backlog=16)

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

[18:05:51.970] [OK] CONNECTION #1 from 10.0.0.3:38246

[18:05:52.523] [OK] CONNECTION #2 from 10.0.0.3:38248

[18:05:53.074] [OK] CONNECTION #3 from 10.0.0.3:38250

SYN-cookies: OFF

[18:05:53.625] [OK] CONNECTION #4 from 10.0.0.3:38252

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

[18:06:51.107] [OK] CONNECTION #5 from 10.0.0.3:38276

[18:06:51.663] [OK] CONNECTION #6 from 10.0.0.3:38278

SYN-cookies: OFF

[18:06:52.240] [OK] CONNECTION #7 from 10.0.0.3:38280

[18:06:52.795] [OK] CONNECTION #8 from 10.0.0.3:38282

[18:06:53.350] [OK] CONNECTION #9 from 10.0.0.3:38284

[18:06:53.905] [OK] CONNECTION #10 from 10.0.0.3:38286

SYN-cookies: OFF

[18:06:54.459] [OK] CONNECTION #11 from 10.0.0.3:38288

[18:06:55.012] [OK] CONNECTION #12 from 10.0.0.3:38290

[18:06:55.564] [OK] CONNECTION #13 from 10.0.0.3:38292

SYN-cookies: OFF

[18:06:56.117] [OK] CONNECTION #14 from 10.0.0.3:38294

[18:06:56.676] [OK] CONNECTION #15 from 10.0.0.3:38296

[18:06:57.225] [OK] CONNECTION #16 from 10.0.0.3:38298

[18:06:57.777] [OK] CONNECTION #17 from 10.0.0.3:38300

SYN-cookies: OFF

[18:06:58.332] [OK] CONNECTION #18 from 10.0.0.3:38302

[18:06:58.886] [OK] CONNECTION #19 from 10.0.0.3:38304

[18:06:59.435] [OK] CONNECTION #20 from 10.0.0.3:38306

[18:06:59.991] [OK] CONNECTION #21 from 10.0.0.3:38308

SYN-cookies: OFF

[18:07:00.544] [OK] CONNECTION #22 from 10.0.0.3:38310

[18:07:01.097] [OK] CONNECTION #23 from 10.0.0.3:38312

[18:07:01.647] [OK] CONNECTION #24 from 10.0.0.3:38314

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

[18:07:21.222] [OK] CONNECTION #25 from 10.0.0.3:38316

[18:07:22.135] [OK] CONNECTION #26 from 10.0.0.3:38318

SYN-cookies: OFF

[18:07:22.995] [OK] CONNECTION #27 from 10.0.0.3:38320

[18:07:23.865] [OK] CONNECTION #28 from 10.0.0.3:38322

SYN-cookies: OFF

[18:07:24.657] [OK] CONNECTION #29 from 10.0.0.3:38324

[18:07:25.622] [OK] CONNECTION #30 from 10.0.0.3:38326

SYN-cookies: OFF

[18:07:26.439] [OK] CONNECTION #31 from 10.0.0.3:38328

[18:07:27.750] [OK] CONNECTION #32 from 10.0.0.3:38330

SYN-cookies: OFF

[18:07:29.349] [OK] CONNECTION #33 from 10.0.0.3:38332

SYN-cookies: OFF

SYN-cookies: OFF

[18:07:32.816] [OK] CONNECTION #34 from 10.0.0.3:38334

SYN-cookies: OFF

[18:07:35.661] [OK] CONNECTION #35 from 10.0.0.3:38336

SYN-cookies: OFF

[18:07:37.000] [OK] CONNECTION #36 from 10.0.0.3:38338

[18:07:37.672] [OK] CONNECTION #37 from 10.0.0.3:38340

[18:07:38.415] [OK] CONNECTION #38 from 10.0.0.3:38342

SYN-cookies: OFF

[18:07:39.131] [OK] CONNECTION #39 from 10.0.0.3:38344
[18:07:39.904] [OK] CONNECTION #40 from 10.0.0.3:38346
SYN-cookies: OFF

[18:07:40.617] [OK] CONNECTION #41 from 10.0.0.3:38348
[18:07:41.395] [OK] CONNECTION #42 from 10.0.0.3:38350
[18:07:42.233] [OK] CONNECTION #43 from 10.0.0.3:38352
SYN-cookies: OFF

[18:07:43.096] [OK] CONNECTION #44 from 10.0.0.3:38354
SYN-cookies: OFF

SYN-cookies: OFF

[18:07:47.243] [OK] CONNECTION #45 from 10.0.0.3:38356
[18:07:47.900] [OK] CONNECTION #46 from 10.0.0.3:38358
SYN-cookies: OFF

[18:07:48.794] [OK] CONNECTION #47 from 10.0.0.3:38360
[18:07:49.575] [OK] CONNECTION #48 from 10.0.0.3:38362
[18:07:50.238] [OK] CONNECTION #49 from 10.0.0.3:38364
SYN-cookies: OFF

[18:07:50.833] [OK] CONNECTION #50 from 10.0.0.3:38366
SYN-cookies: OFF

[18:07:54.513] [OK] CONNECTION #51 from 10.0.0.3:38370
SYN-cookies: OFF

[18:07:55.165] [OK] CONNECTION #52 from 10.0.0.3:38372
[18:07:55.937] [OK] CONNECTION #53 from 10.0.0.3:38374
SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

[18:08:25.754] [OK] CONNECTION #54 from 10.0.0.3:38384

[18:08:26.343] [OK] CONNECTION #55 from 10.0.0.3:38386

[18:08:27.020] [OK] CONNECTION #56 from 10.0.0.3:38388

SYN-cookies: OFF

[18:08:27.581] [OK] CONNECTION #57 from 10.0.0.3:38390

[18:08:28.228] [OK] CONNECTION #58 from 10.0.0.3:38392

[18:08:28.813] [OK] CONNECTION #59 from 10.0.0.3:38394

SYN-cookies: OFF

[18:08:29.377] [OK] CONNECTION #60 from 10.0.0.3:38396

[18:08:29.938] [OK] CONNECTION #61 from 10.0.0.3:38398

[18:08:30.515] [OK] CONNECTION #62 from 10.0.0.3:38400

[18:08:31.103] [OK] CONNECTION #63 from 10.0.0.3:38402

SYN-cookies: OFF

[18:08:31.682] [OK] CONNECTION #64 from 10.0.0.3:38404

[18:08:32.302] [OK] CONNECTION #65 from 10.0.0.3:38406

[18:08:32.905] [OK] CONNECTION #66 from 10.0.0.3:38408

SYN-cookies: OFF

[18:08:33.476] [OK] CONNECTION #67 from 10.0.0.3:38410

[18:08:34.045] [OK] CONNECTION #68 from 10.0.0.3:38412

[18:08:34.611] [OK] CONNECTION #69 from 10.0.0.3:38414

[18:08:35.184] [OK] CONNECTION #70 from 10.0.0.3:38416

SYN-cookies: OFF

[18:08:35.752] [OK] CONNECTION #71 from 10.0.0.3:38418

[18:08:36.366] [OK] CONNECTION #72 from 10.0.0.3:38420

[18:08:36.974] [OK] CONNECTION #73 from 10.0.0.3:38422

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

[18:09:09.013] [OK] CONNECTION #74 from 10.0.0.3:38436
SYN-cookies: OFF

[18:09:09.950] [OK] CONNECTION #75 from 10.0.0.3:38438
[18:09:10.650] [OK] CONNECTION #76 from 10.0.0.3:38440
[18:09:11.436] [OK] CONNECTION #77 from 10.0.0.3:38442
SYN-cookies: OFF

[18:09:12.375] [OK] CONNECTION #78 from 10.0.0.3:38444
[18:09:13.395] [OK] CONNECTION #79 from 10.0.0.3:38446
SYN-cookies: OFF

[18:09:14.125] [OK] CONNECTION #80 from 10.0.0.3:38448
[18:09:15.059] [OK] CONNECTION #81 from 10.0.0.3:38450
SYN-cookies: OFF

[18:09:15.970] [OK] CONNECTION #82 from 10.0.0.3:38452
[18:09:16.552] [OK] CONNECTION #83 from 10.0.0.3:38454
[18:09:17.304] [OK] CONNECTION #84 from 10.0.0.3:38456
SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: OFF

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

[18:10:03.943] [OK] CONNECTION #85 from 10.0.0.3:38466

SYN-cookies: ON

SYN-cookies: ON

[18:10:07.238] [OK] CONNECTION #86 from 10.0.0.3:38468

SYN-cookies: ON
[18:10:09.250] [OK] CONNECTION #87 from 10.0.0.3:38470
SYN-cookies: ON

[18:10:11.050] [OK] CONNECTION #88 from 10.0.0.3:38472
[18:10:11.854] [OK] CONNECTION #89 from 10.0.0.3:38474
SYN-cookies: ON

[18:10:12.634] [OK] CONNECTION #90 from 10.0.0.3:38476
[18:10:13.394] [OK] CONNECTION #91 from 10.0.0.3:38478
[18:10:14.082] [OK] CONNECTION #92 from 10.0.0.3:38480
SYN-cookies: ON

[18:10:14.703] [OK] CONNECTION #93 from 10.0.0.3:38482
[18:10:15.422] [OK] CONNECTION #94 from 10.0.0.3:38484
[18:10:16.124] [OK] CONNECTION #95 from 10.0.0.3:38486
SYN-cookies: ON

[18:10:16.727] [OK] CONNECTION #96 from 10.0.0.3:38488
[18:10:17.429] [OK] CONNECTION #97 from 10.0.0.3:38490
[18:10:18.334] [OK] CONNECTION #98 from 10.0.0.3:38492
SYN-cookies: ON

[18:10:19.059] [OK] CONNECTION #99 from 10.0.0.3:38494
[18:10:19.747] [OK] CONNECTION #100 from 10.0.0.3:38496
[18:10:20.634] [OK] CONNECTION #101 from 10.0.0.3:38498
SYN-cookies: ON

[18:10:21.362] [OK] CONNECTION #102 from 10.0.0.3:38500
[18:10:22.218] [OK] CONNECTION #103 from 10.0.0.3:38502
SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

SYN-cookies: ON

Activating cookies

```
root@mininet-vm:/home/mininet/ECE436_networksecurity_project# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
```